View Config Manual

A comprehensive guide to authoring **view config JSON** files for the UiApi package. These files drive the Component Config Service (CCS), which assembles UI payloads — headers, form fields, filters, data links, toolbar settings, and more — all from a single JSON file per model.

Table of Contents

1. How It Works
2. File Naming & Location
3. Architecture: Views & Components
4. The View Block
5. Component: table
6. Component: form
7. Component: toolbar
8. Component: filterSection
9. Component: meta
10. Column Customizations
11. The Override Mechanism
12. External JS Functions
13. Localization
14. noModel Mode
15. Validation Rules
16. Complete Examples
17. Quick Checklist

---

1. How It Works

CCS reads your view config JSON, merges it with the model's `apiSchema()` and built-in component config templates, and returns a fully assembled UI payload.

```
View Config JSON  ──────────┐
```

```
                                  ├ →  CCS  →  JSON Response
    Model apiSchema()  ──────────┤          (componentSettings, headers,
filters, etc.)
    Component Templates  ────────┘
    (table.json, form.json, etc.)
```

---

2. File Naming & Location

Location

View config files live in the directory specified by `uiapi.view_configs_path` in `config/uiapi.php`. Default: `app/Services/viewConfigs/`.

Naming Rules

The filename must be the **lowercased, no-separator** version of the model name, with a `.json` extension.

Model Name	Filename
`Person`	`person.json`
`CForm`	`cform.json`
`LegalAid`	`legalaid.json`
`CriminalRecord`	`criminalrecord.json`

noModel Naming

For noModel configs (no backing Eloquent model), the filename is still just a lowercase identifier. There is no naming prefix requirement — just make sure the name is unique so CCS can find it.

---

3. Architecture: Views & Components

A view config JSON has a **two-tier structure**:

1. **View blocks** — Keys containing `"View"` in the name (e.g. `listView`, `detailView`). These define *which components* a particular screen uses and configure view-level settings like language support.
2. **Component definitions** — Top-level keys that do NOT contain `"View"` (e.g. `table`, `form`, `toolbar`, `filterSection`, `meta`). These define the overrides and settings for each individual component.

```
    {
        "listView": {                    // ← View block (references
components)
            "components": { ... },
            "lang": ["en", "dv"]
        },
        "table": { ... },               // ← Component definition
(overrides table.json template)
        "form": { ... },                // ← Component definition
(overrides form.json template)
        "toolbar": { ... },             // ← Component definition
        "filterSection": { ... },       // ← Component definition
        "meta": { ... }                 // ← Component definition
    }
```

CCS distinguishes views from components by checking if the key contains `"View"`. This means:

- `listView`, `detailView`, `listView2` — all treated as views.
- `table`, `form`, `toolbar`, `filterSection`, `meta` — all treated as component definitions.

Why This Separation?

This two-tier approach allows:

- **Reuse** — Multiple views can reference the same component definitions.
- **Cross-model references** — A view can reference a component from a different model's config.
- **Clean separation** — View-level concerns (language, which components to show) are separate from component-level concerns (columns, customizations, form fields).

---

4. The View Block

A view block declares which components a particular screen uses and provides view-level settings.

Structure

```
"listView": {
    "components": {
        "table": "CForm/table",
        "form": "CForm/form",
        "toolbar": "CForm/toolbar",
        "filterSection": "CForm/filterSection",
        "meta": "CForm/meta"
    },
    "lang": ["en", "dv"]
}
```

`components` (required)

A map of **alias → reference**. The alias is the component role name (`table`, `form`, etc.) and the reference tells CCS where to find the component definition.

Reference format:

Format	Meaning	Example
`"ModelName/componentKey"`	Self-reference (same model) or cross-model reference	`"CForm/table"`
`"componentKey"`	Direct reference to a root-level key in the same file	`"table"`

`lang` (required on view or component)

Array of supported language codes. CCS validates the request `lang` parameter against this list. If the requested language isn't in the array, CCS returns an empty response.

```
"lang": ["en", "dv"]
```

Where to define `lang`: You can define it on the view block, on individual component definitions, or both. If a component definition doesn't have `lang`, it inherits it from the view that references it. If a component has its own `lang`, that takes precedence.

Supported codes: `"en"` (English), `"dv"` (Dhivehi).

Multiple Views

You can define multiple views in one file:

```
{
    "listView": {
        "components": {
            "table": "CForm/table",
            "toolbar": "CForm/toolbar"
        },
        "lang": ["en", "dv"]
    },
    "detailView": {
        "components": {
            "form": "CForm/form",
            "meta": "CForm/meta"
        },
        "lang": ["en"]
    },
    "table": { ... },
    "form": { ... },
    "toolbar": { ... },
    "meta": { ... }
}
```

---

5. Component: table

The table component drives the data table — headers, pagination, data link, action buttons, delete confirmations, and column display.

Base Template (`ComponentConfigs/table.json`)

```
{
    "table": {
        "headers": "on",
```

```
            "filters": "off",
            "pagination": "on",
            "datalink": "on",
            "button": ["create", "edit", "delete"],
            "showActions": true,
            "actions": {
                "showView": true,
                "showEdit": true,
                "showDelete": true
            }
        }
    }
```

The base template defines the *skeleton*. Each key uses a **tri-state** pattern:

Value	Behavior
`"on"`	CCS auto-generates this section from the model's schema
`"off"`	Section is omitted from the payload
Any other value	Passed through as-is

So `"headers": "on"` means CCS will automatically build the headers array from `apiSchema()` columns + any column customizations you define.

Table Component Keys

Define these keys in your root-level `"table": { ... }` block to override or extend the template.

`columns` (array of strings)

Which columns to show in the table. Each entry is either a bare column name (`"ref_num"`) or a dot-notation relation token (`"createdby.first_name_div"`).

```
    "columns": [
        "id",
        "uuid",
        "ref_num",
        "summary",
        "status",
        "createdby.first_name_div"
    ]
```

Dot notation: `"relation.field"` tells CCS to:

1. Load the related model's `apiSchema()` to get header metadata.
2. Automatically add the relation to the `with` parameter in the data link.

If `columns` is omitted, CCS uses all non-hidden columns from `apiSchema()`.

`columnCustomizations` (object)

Overrides for how individual columns are displayed. See [Section 10: Column Customizations] for full details.

```
"columnCustomizations": {
    "ref_num": {
        "width": "auto",
        "align": "center",
        "sortable": true
    },
    "status": {
        "displayType": "chip",
        "chip": { ... },
        "sortable": true,
        "inlineEditable": true
    }
}
```

`per_page` (integer)

Default number of rows per page. Overridable at request time via query parameter.

```
"per_page": 15
```

Default if omitted: `25`.

`actions` (object)

Controls which CRUD action buttons appear for each row.

```
"actions": {
    "showView": false,
    "showEdit": true,
    "showDelete": true
}
```

Values defined here are **merged** with the template defaults. So to hide the view button, you only need to set `"showView": false` — the others remain as-is.

`delete` (object)

Configuration for the delete confirmation dialog.

```
"delete": {
    "title": {
        "en": "Confirm Deletion",
        "dv": "ڏُعَرَمْڏَرْ سِرِسْرُڏُ"
    },
    "subtitle": {
        "en": "Are you sure you want to delete this item?",
        "dv": "..."
    },
    "message": {
        "ref_num": { "en": "ref", "dv": "مَدَرِيعِ" },
        "uuid": { "en": "UUID", "dv": "ڏُرِمِڏُعِ" }
    },
    "crudLink": "gapi/CForm"
}
```

Key	Type	Description
`title`	`{en, dv}`	Dialog title
`subtitle`	`{en, dv}`	Confirmation message
`message`	`object`	Map of field → label pairs shown as summary in the dialog
`crudLink`	`string`	DELETE endpoint (optional; auto-generated from model name if omitted)

`functions` (object)

External JS function definitions for custom column data. See [Section 12: External JS Functions](#).

```
"functions": {
    "customeColumnData": {
        "file": "misc.js",
        "function": "customeColumnData"
    }
}
```

`filters` (array of strings)

Column keys to expose as filters within the table component. In the base template, table filters are `"off"` by default — this key only takes effect when the template directive is changed.

`lang` (array of strings)

Override the language support for this component specifically. If omitted, inherited from the view block.

---

6. Component: form

The form component drives create/edit forms — field layout, validation, groups, search fields, and conditional visibility.

Base Template (`ComponentConfigs/form.json`)

```
{
    "form": {
        "fields": "on",
        "rules": "on",
        "createLink": "off",
        "crudLink": "on",
        "updateLink": "on",
        "numberOfColumns": 3,
        "button": ["create", "edit"]
    }
}
```

When `"fields": "on"`, CCS auto-generates form fields from columns that have `"formField": true` in `apiSchema()`. When you provide explicit `fields` in your form config, they override and restrict the auto-generated set.

Form Component Keys

`createTitle` / `editTitle` ({en, dv})

Titles displayed at the top of the form in create and edit modes.

```
    "createTitle": { "en": "Create Form", "dv": "ؤَدَمْ" },
    "editTitle": { "en": "Edit Form", "dv": "ھَڪَرَڏُ" }
```

`banner` (object)

An optional informational banner shown at the top of the form.

```
    "banner": {
        "title": { "en": "Form Details", "dv": "..." },
        "message": { "en": "Please fill out the form below...", "dv":
"..." },
        "color": "info",
        "icon": "information-circle",
        "variant": "tonal"
    }
```

Key	Type	Description
`title`	`{en, dv}`	Banner heading
`message`	`{en, dv}`	Banner body text
`color`	`string`	Color token: `"info"`, `"success"`, `"warning"`, `"error"`
`icon`	`string`	Icon name
`variant`	`string`	Display variant: `"tonal"`, `"flat"`, `"outlined"`, etc.

`groups` (array of objects)

Forms are organized into named **groups** (sections). Each group gets its own heading and column layout.

```
"groups": [
    {
        "name": "Search",
        "title": { "en": "Search", "dv": "ڑﷺ" },
        "numberOfColumns": 2
    },
    {
        "name": "BasicInfo",
        "condition": "SearchResults || formType === 'edit'",
        "title": { "en": "Basic Information", "dv": "ﷺﷺﷺﷺ" },
        "numberOfColumns": 3
    },
    {
        "name": "Submission",
        "condition": "SearchResults || formType === 'edit'",
        "title": { "en": "Submission", "dv": "ﷺﷺﷺ ﷺﷺﷺ" },
        "numberOfColumns": 2
    }
]
```

Key	Type	Required	Description
`name`	`string`	Yes	Unique identifier. Fields reference this via their `group` key. Must be unique across all groups.
`title`	`{en, dv}`	Yes	Section heading. Should be localized.
`numberOfColumns`	`integer`	No	Grid columns for this group (overrides the form-level `numberOfColumns`).
`condition`	`string`	No	A JavaScript expression referencing variables from `variables`. The group is only visible when this evaluates to truthy.

`fields` (array of objects)

When you provide `fields`, you are **overriding** the auto-generated form fields. CCS merges your field overrides with the schema-generated fields by matching on the `key` property. Fields you list are included; fields you don't list may be excluded (depending on how the override merge works).

Each field object requires at minimum a `key`:

```
"fields": [
    {
        "key": "nid_pp",
        "group": "Search",
        "disabled": "mode == 'edit' ? true : false",
        "columnSpan": 2,
        "inputType": "search",
        "submitUrl": "gapi/person/",
        "submitMethod": "get",
        "NotDbField": true,
        "lang": "en",
        "submitParam": "lang=${language}&filter",
        "events": {
            "results": "handleSearchResults"
        }
    },
    {
        "key": "ref_num",
        "disabled": "mode == 'edit' ? true : false",
        "group": "BasicInfo",
        "rules": "[rules.required]"
    },
    { "key": "client_ref_number", "group": "BasicInfo" },
    {
        "key": "summary",
        "inputType": "textarea",
        "group": "BasicInfo",
        "columnSpan": 3,
        "hidden": false
    },
    {
        "key": "entry_type",
        "group": "Submission",
        "inputType": "select",
        "mode": "url",
        "url": "http://example.com/api/entry-types",
        "itemTitle": "itemTitleDv",
        "itemValue": "itemValue"
    },
    {
        "key": "submitted_at",
        "group": "Submission",
        "format": "dd/MM/yyyy"
    },
    {
        "key": "file_upload",
        "label": { "en": "Upload File", "dv": "ف̊ؤ̊ڊ̊ڎ̊ ؤ̊ڑ̊ڑ̊ڑ̊ڕ̊" },
        "inputType": "file",
        "columnSpan": 3
    }
]
```

Field Keys Reference

Key	Type	Required	Description
`key`	`string`	Yes	Column name. Must match an `apiSchema()` key (unless it's a non-DB field).
`group`	`string`	Yes	Which group this field belongs to. Must match a `groups[].name`.
`inputType`	`string`	No	Override the auto-detected input type. See Input Types.
`label`	`{en, dv}`	No	Override the schema label.
`columnSpan`	`integer`	No	Number of grid columns to span (default: 1).
`hidden`	`boolean`	No	Hide the field from the UI.
`disabled`	`string` or `boolean`	No	Disable condition. String values are JS expressions evaluated by the frontend.
`placeholder`	`string`	No	Placeholder text for the input.
`format`	`string`	No	Display format (e.g. `"dd/MM/yyyy"` for date fields).
`rules`	`string`	No	Frontend validation rules expression (e.g. `"[rules.required]"`).
`condition`	`string`	No	JS expression for conditional visibility (like group condition).
`color`	`string`	No	Color token for certain input types (e.g. checkbox).
`NotDbField`	`boolean`	No	`true` if this field does not map to a database column.
`lang`	`string`	No	Restrict this field to a specific language (`"en"` or `"dv"`).
`events`	`object`	No	Map of event name → function name. See Functions & Events.

Input Types

`inputType` Value	Description	Extra Keys Required
`"text"` / `"textField"`	Standard text input	—
`"textarea"`	Multi-line text input	—

`inputType` Value	Description	Extra Keys Required
`"number"` / `"numberField"`	Numeric input	—
`"checkbox"`	Boolean checkbox	—
`"date"` / `"datepicker"`	Date picker	`format` (optional)
`"select"`	Dropdown select	`mode`, `items`/`url`, `itemTitle`, `itemValue` (see Select in Forms)
`"search"`	Search-as-you-type field	`submitUrl` (required), `submitMethod`, `submitParam`, `events`
`"file"`	File upload	—

Auto-detection: If you omit `inputType`, CCS derives it from the column's `type` in `apiSchema()`:

Schema `type`	Default `inputType`
`string`	`textField`
`number`	`numberField`
`boolean`	`checkbox`
`date`	`datepicker`

Select in Forms

When `inputType` is `"select"`, you configure the dropdown directly in the field object:

```
{
    "key": "entry_type",
    "group": "Submission",
    "inputType": "select",
    "mode": "url",
    "url": "http://example.com/api/entry-types",
    "itemTitle": "itemTitleDv",
    "itemValue": "itemValue"
}
```

Or using self mode with static items:

```
    {
        "key": "status",
        "group": "Submission",
        "inputType": "select",
        "mode": "self",
        "items": [
            { "itemTitleEn": "Draft", "itemTitleDv": "ڊرافٽ",
"itemValue": "draft" },
            { "itemTitleEn": "Submitted", "itemTitleDv": "سَبْمِٽْ",
"itemValue": "submitted" }
        ],
        "itemTitle": "itemTitleEn",
        "itemValue": "itemValue"
    }
```

> **Note:** Select configuration for form fields can also come from the model's `apiSchema()` (via the `select` or `filterable` key on the column definition). The viewConfig field-level select config takes precedence.

Search Fields

Search fields (`"inputType": "search"`) allow users to search an external endpoint and populate form data from the results.

```
    {
        "key": "nid_pp",
        "group": "Search",
        "inputType": "search",
        "submitUrl": "gapi/person/",
        "submitMethod": "get",
        "submitParam": "lang=${language}&filter",
        "events": {
            "results": "handleSearchResults"
        }
    }
```

Key	Type	Required	Description
`submitUrl`	`string`	Yes	The API endpoint to send search queries to.
`submitMethod`	`string`	No	HTTP method (`"get"` or `"post"`). Default: `"get"`.

Key	Type	Required	Description
`submitParam`	`string`	No	Query parameter template. Supports `${language}` interpolation.
`events.results`	`string`	No	Function name to handle the search results. Must be defined in `functions`.

Functions & Events

Field-level `events` map event names to function names. The function names must be defined in the form's `functions` block.

```
"fields": [
    {
        "key": "nid_pp",
        "events": { "results": "handleSearchResults" }
    }
],
"functions": {
    "handleSearchResults": {
        "file": "misc.js",
        "function": "handleSearchResults"
    }
}
```

The validator enforces that every event handler name exists in the `functions` block.

`variables` (object)

Reactive variables used for conditional visibility of groups and fields. The frontend manages their state; the view config just declares initial values.

```
"variables": {
    "draftingEnabled": true,
    "SearchResults": true
}
```

Referenced by `condition` in groups:

```
{
```

```
        "name": "BasicInfo",
        "condition": "SearchResults || formType === 'edit'"
    }
```

`functions` (object)

External JavaScript functions. See Section 12 for full syntax.

`lang` (array)

Override language support for this component. If omitted, inherited from the view.

---

7. Component: toolbar

The toolbar component drives the top toolbar — title, search/filter buttons, and quick filters.

Base Template (`ComponentConfigs/toolbar.json`)

```
{
    "toolbar": {
        "filters": "on",
        "searchAll": "on",
        "buttons": [
            {
                "type": "search",
                "color": "primary",
                "icon": "magnifying-glass",
                "variant": "contained"
            },
            {
                "type": "Clear",
                "label": "Clear",
                "color": "secondary"
            }
        ]
    }
}
```

Toolbar Component Keys

`title` ({en, dv})

The toolbar heading.

```
"title": { "en": "Forms", "dv": "ﷺﷺﷺ
```

```
            "key": "ref_num",
            "label": { "en": "Search by ID", "dv": "ތަރުތީބު ބިން ހުރ" },
            "color": "primary",
            "prependIcon": "search"
        }
    }
```

`lang` (array)

Override language support.

---

8. Component: filterSection

The filter section component displays an expandable filter panel with column-based filters and action buttons.

Base Template (`ComponentConfigs/filterSection.json`)

```
    {
        "filterSection": {
            "buttons": [
                {
                    "type": "submit",
                    "color": "primary",
                    "prependIcon": "magnifying-glass",
                    "variant": "contained"
                },
                {
                    "type": "Clear",
                    "label": "x",
                    "appendIcon": "funnel",
                    "color": "secondary"
                },
                {
                    "type": "Close",
                    "prependIcon": "x-mark",
                    "label": "Close",
                    "color": "gray"
                }
            ],
            "filters": "on"
        }
    }
```

FilterSection Component Keys

`filters` (array of strings)

Column keys to expose as filter fields in the filter section.

```
"filters": ["status"]
```

CCS builds filter metadata (type, key, label, items/url) from the column's `apiSchema()` definition (or from `columnCustomizations` in the filterSection).

`buttons` (array of strings)

Filter the template buttons to only the listed types.

```
"buttons": ["submit", "clear"]
```

`columnCustomizations` (object)

Override or extend column configuration *specifically* for the filter section. This is commonly used to configure select dropdowns for filter fields.

```
"columnCustomizations": {
    "status": {
        "width": "180px",
        "displayType": "chip",
        "sortable": true,
        "inlineEditable": true,
        "inputType": "select",
        "select": {
            "type": "select",
            "mode": "self",
            "items": [
                {
                    "itemTitleEn": "Draft",
                    "itemTitleDv": "ڊرافٽ",
                    "itemValue": "draft"
                },
                {
```

```
                        "itemTitleEn": "Submitted",
                        "itemTitleDv": "ސަބْމِٹْ ކުރެވުނު",
                        "itemValue": "submitted"
                    }
                ]
            }
        }
    }
```

The `select` configuration follows the same rules as `apiSchema()` select configs. See `select` modes below.

Select Modes in Filters

Mode	Description	Required Keys
`"self"`	Static items defined inline	`items` (array of objects)
`"url"`	Fetch options from a custom URL	`url` (string)
`"relation"`	Auto-generate URL from an Eloquent relationship	`relationship` (string, optional — CCS can guess from column name)

Self mode example:

```
    "select": {
        "mode": "self",
        "items": [
            { "itemTitleEn": "Draft", "itemTitleDv": "ڊﷺ",
"itemValue": "draft" }
        ]
    }
```

URL mode example:

```
    "select": {
        "mode": "url",
        "url": "http://example.com/api/options"
    }
```

Relation mode example:

```
    "select": {
```

```
        "mode": "relation",
        "relationship": "country",
        "itemTitle": { "en": "name_eng", "dv": "name_div" },
        "itemValue": "id"
    }
```

`lang` (array)

Override language support.

---

9. Component: meta

The meta component provides metadata — primarily the CRUD link for create/update operations.

Base Template (`ComponentConfigs/meta.json`)

```
    {
        "meta": {
            "crudLink": "on"
        }
    }
```

When `"crudLink": "on"`, CCS auto-generates `gapi/{ModelName}` as the CRUD endpoint.

Meta Component Keys

`crudLink` (string)

Override the auto-generated CRUD link with a custom URL.

```
    "meta": {
        "crudLink": "http://external-
api.example.com/api/gapi/CriminalRecordRequest"
    }
```

When a custom string is provided (not `"on"` or `"off"`), it's passed through as-is.

`lang` (array)

Override language support.

Any Custom Keys

The meta component is a good place for arbitrary pass-through metadata that the frontend needs:

```
"meta": {
    "crudLink": "on",
    "customSetting": "value"
}
```

> **Note:** Custom keys only pass through if `allow_custom_component_keys` is `true` in `config/uiapi.php`.

---

10. Column Customizations

Column customizations override or extend schema-derived header properties. They control how columns appear in the table — width, display type, sorting, inline editing, and more.

Where to Define

Column customizations can be defined in **two places**:

1. **Inside a component definition** — Applies to that component only.
2. **(noModel only) In `columnCustomizations` within the component block** — Same effect.

When both exist, they are **deep-merged** (component-specific wins on conflict).

Accepted Keys

Key	Type	Description
`title`	`{en, dv}` or `string`	Override the header title
`label`	`{en, dv}` or `string`	Override the header label (used for custom columns)
`hidden`	`boolean`	Hide/show in table headers
`sortable`	`boolean`	Enable/disable column sorting
`width`	`string`	Column width (e.g. `"180px"`, `"200px"`, `"auto"`)
`align`	`string`	Text alignment (`"center"`, `"left"`, `"right"`)
`headerAlign`	`string`	Header text alignment
`displayType`	`string`	How the cell renders (see Display Types)
`type`	`string`	Data type hint (`"string"`, `"number"`, `"boolean"`, `"date"`)
`inlineEditable`	`boolean`	Allow inline editing in the table
`editable`	`boolean`	Alias for `inlineEditable`
`displayProps`	`object`	Display configuration (legacy; prefer the display-type-named key)
`order`	`integer`	0-based insertion position for header reordering
`inputType`	`string`	Input type for filter usage (e.g. `"select"`)
`select`	`object`	Select/dropdown configuration

Display Types

`displayType`	Description	Requires Config?
`"text"`	Plain text (default)	No
`"chip"`	Colored badge/tag	Yes — `chip` sub-key
`"date"`	Formatted date	No
`"checkbox"`	Boolean checkbox	No

`displayType`	Description	Requires Config?
`"select"`	Dropdown display	Yes — `displayProps` or `select` sub-key
`"custom"`	Custom rendering via JS function	Yes — `columnData` key
`"link"`	Clickable link	No
`"image"`	Image display	No
`"badge"`	Badge display	No
`"html"`	Raw HTML rendering	No

Chip Configuration

When `displayType` is `"chip"`, provide a `chip` object where each key is a possible **field value** and the value is the chip's appearance config:

```
"status": {
    "displayType": "chip",
    "chip": {
        "draft": {
            "size": "sm",
            "label": { "en": "Draft", "dv": "ڤْرَفْ" },
            "color": "secondary",
            "prependIcon": "document-text"
        },
        "submitted": {
            "size": "sm",
            "label": { "en": "Submitted", "dv": "ﷺ" },
            "color": "primary",
            "prependIcon": "paper-airplane"
        },
        "rejected": {
            "size": "sm",
            "label": { "en": "Rejected", "dv": "ﷺ" },
            "color": "error",
            "prependIcon": "x-circle"
        },
        "accepted": {
            "size": "sm",
            "label": { "en": "Accepted", "dv": "ﷺ" },
            "color": "success",
            "prependIcon": "check-circle"
        }
    }
}
```

Chip Option Key	Type	Description
`size`	`string`	Size variant: `"sm"`, `"md"`, `"lg"`
`label`	`{en, dv}`	Display label (auto-resolved to request language)
`color`	`string`	Color token: `"primary"`, `"secondary"`, `"success"`, `"error"`, `"warning"`
`prependIcon`	`string`	Icon name displayed before the label

Custom Columns

You can add entirely **new columns** that don't exist in the model's schema. These appear as extra headers in the table.

```
"columnCustomizations": {
    "New_ref": {
        "width": "400px",
        "label": { "en": "New Column", "dv": "ﷺ" },
        "columnData": "customeColumnData()",
        "order": 3,
        "displayType": "custom"
    }
}
```

Key	Type	Description
`label`	`{en, dv}` or `string`	Required for custom columns (otherwise title-cased from key)
`columnData`	`string`	JS expression or function call for dynamic cell data
`order`	`integer`	Position in the header list
`displayType`	`string`	Typically `"custom"` for JS-rendered columns

The `columnData` value references a function defined in the `functions` block. CCS extracts the function body and inlines it.

Header Reordering (`order`)

The `order` key sets a **0-based insertion position** for the header:

- Headers without `order` keep their natural sequence.
- Headers with `order` are pulled out, sorted by `order` value, then spliced in at the requested index.
- Out-of-range values are clamped.
- The `order` key is stripped from the final payload.

```
    "columnCustomizations": {
        "status": { "order": 0 },
        "ref_num": { "order": 2 },
        "New_ref": { "label": { "en": "Custom" }, "order": 3,
"displayType": "custom" }
    }
```

---

11. The Override Mechanism

Understanding how CCS merges your view config overrides with base component templates is essential for advanced usage.

How It Works

1. CCS loads the **base component template** (e.g. `table.json`).
2. Each template directive is processed (e.g. `"headers": "on"` → builds the headers array).
3. Your component block overrides are applied on top via `applyOverridesToSection()`.

Override Behaviors by Value Type

Override Value	What Happens
`"off"` (string)	**Removes** the matching key from the payload entirely
Other scalar (string, number, bool)	**Replaces** the existing value
Array of strings (e.g. `["search", "clear"]`)	**Filters** the template array — keeps only items whose `type`, `key`, `name`, or `label` matches one of the strings (case-insensitive)

Override Value	What Happens
Array of objects (for `fields`)	**Replaces** — your array becomes the fields list, merged by `key` with schema-generated fields
Array of objects (for other keys)	**Merges** — matches existing items by `key`, merges properties; appends unmatched items
Object	**Deep-merges** with the existing object

Practical Examples

Filtering buttons:

Template has three buttons. You only want "search" and "clear":

```
"buttons": ["search", "clear"]
```

CCS scans the template's button array and keeps only items where `type` matches `"search"` or `"clear"`.

Removing a section:

To hide pagination from the table payload:

```
"pagination": "off"
```

Adding custom keys:

To pass arbitrary data to the frontend, set `allow_custom_component_keys: true` in `config/uiapi.php`, then add any key:

```
"variables": {
    "draftingEnabled": true
}
```

Without `allow_custom_component_keys`, unknown keys are silently ignored.

Internal Keys (Stripped from Payload)

These keys are consumed by CCS during processing and **never appear** in the final API response:

- `columnCustomizations`
- `columnsSchema`
- `noModel`
- `columns`
- `per_page`
- `filters`
- `lang`

---

12. External JS Functions

Components can reference JavaScript functions stored in external `.js` files. CCS extracts the function body at runtime and inlines it into the payload as a string.

Configuration

Setting	Default
Config key	`uiapi.js_scripts_path`
Default path	`app/Services/jsScripts/`

Syntax

Inside any component block, add a `functions` object:

```
    "functions": {
        "handleSearchResults": {
            "file": "misc.js",
            "function": "handleSearchResults"
        },
```

```
        "customeColumnData": {
            "file": "misc.js",
            "function": "customeColumnData"
        }
    }
```

Key	Type	Required	Description
`file`	`string`	Yes	JS filename in the `js_scripts_path` directory
`function`	`string`	Yes	Function name to extract (must be a named `function` declaration)

Inline string functions are also supported:

```
    "functions": {
        "simpleHandler": "console.log('hello');"
    }
```

How Extraction Works

CCS reads the JS file, finds `function functionName(...) {`, then uses **brace-counting** to extract everything between the opening `{` and closing `}`. Nested braces are handled correctly.

The extracted body is returned as a single-line string with whitespace trimmed.

JS File Example

`app/Services/jsScripts/misc.js`:

```
    function handleSearchResults() {
        console.log("Responses:", response);
        if (response && response.data && response.data.length > 0) {
            formData.client_ref_number = response.data[0].nid_pp;
            formVariables.SearchResults = true;
        }
    }

    function customeColumnData(item) {
        const componentData = {
```

```
            type: 'chips',
            chips: [
                { size: 'small', label: item.ref_num, color: 'success'
},
                { size: 'small', label: item.uuid, color: 'error' }
            ]
        };
        return componentData;
    }
```

Linking Events to Functions

In form fields, the `events` key maps event names to function names. The validator checks that every referenced function name exists in the `functions` block.

```
    "fields": [
        {
            "key": "nid_pp",
            "events": { "results": "handleSearchResults" }
        }
    ],
    "functions": {
        "handleSearchResults": { "file": "misc.js", "function":
"handleSearchResults" }
    }
```

If you reference a function name in `events` that doesn't exist in `functions`, the validator issues a warning.

---

13. Localization

Language Selection

CCS resolves the language from the `?lang=` query parameter (default: `"dv"`).

Localized Values

Any key that accepts `{en, dv}` is collapsed to a single string based on the request language:

```
"title": { "en": "Forms", "dv": "ﯞﯞﯞﯞﯞﯞﯞﯞ" }
```

With `?lang=en`, the response contains `"title": "Forms"`.

Auto-Collapsed Keys

These payload keys are automatically collapsed from `{en, dv}` objects to a single string:

- `createTitle`
- `editTitle`
- `title`
- `label`

Column Language Filtering

Columns that have a `lang` array in their `apiSchema()` definition are only included when the request language matches:

```
{
    "first_name_eng": {
        "lang": ["en"]
    }
}
```

This column is excluded when `?lang=dv`. Columns with `"lang": ["en", "dv"]` appear for both languages. Columns without a `lang` array appear for all languages.

Language Fallback

When resolving localized values, CCS uses this fallback chain:

1. Requested language (e.g. `en`)
2. Opposite language (e.g. `dv`)
3. First available value

14. noModel Mode

By default, CCS resolves an Eloquent model from the model name and reads its `apiSchema()` for column definitions. **noModel mode** skips model resolution entirely — you provide the column schema inline in the view config JSON.

When to Use noModel

- The data comes from an **external API** (not a local database).
- There is **no Eloquent model** for this data.
- You want to display data from a **third-party service**.
- You need a **fully frontend-driven** table/form with no server-side model.

Enabling noModel

Set `"noModel": true` on each component definition that should operate without a model:

```
{
    "listView": {
        "noModel": true,
        "components": {
            "table": "criminalrecord/table",
            "form": "criminalrecord/form",
            "filterSection": "criminalrecord/filterSection",
            "toolbar": "criminalrecord/toolbar",
            "meta": "criminalrecord/meta"
        }
    },
    "table": {
        "noModel": true,
        "lang": ["en", "dv"],
        ...
    },
    "form": {
        "noModel": true,
        "lang": ["en", "dv"],
        ...
    }
}
```

> **Important:** `noModel` must be set on **each component definition** that needs it, not just the view. The view-level `noModel` signals the view itself, but CCS checks `noModel` per-component when building payloads.

Required: `columnsSchema`

In noModel mode, there is no `apiSchema()` to read from. You must provide a `columnsSchema` object that acts as the replacement. This is required for components that consume column data: **table**, **toolbar**, **filterSection**.

```
    "table": {
        "noModel": true,
        "lang": ["en", "dv"],
        "columnsSchema": {
            "id": {
                "hidden": false,
                "key": "id",
                "label": { "dv": "އައިޑީ", "en": "Id" },
                "lang": ["en", "dv"],
                "type": "number",
                "displayType": "text",
                "formField": false,
                "sortable": true
            },
            "ref_num": {
                "hidden": false,
                "key": "ref_num",
                "label": { "dv": "ތަފުސީލުގެ ނަންބަر", "en": "Reference
Number" },
                "lang": ["en", "dv"],
                "type": "text",
                "displayType": "text",
                "formField": false,
                "sortable": true
            },
            "status": {
                "hidden": false,
                "key": "status",
                "label": { "dv": "ސްޓޭޓަސް", "en": "Status" },
                "lang": ["en", "dv"],
                "type": "text",
                "displayType": "text",
                "formField": false,
                "sortable": true
            }
        },
        "columns": ["id", "ref_num", "status"],
        ...
    }
```

The `columnsSchema` entries use the same format as `apiSchema()` column definitions. See the apiSchema Reference Manual for the full key reference.

Required: `columns`

Even in noModel mode, you still need a `columns` array to define ordering:

```
    "columns": ["id", "ref_num", "person_name", "permanent_address",
"details", "person__id", "status"]
```

Required: `lang`

In noModel mode, `lang` must be explicitly set on each component (it can't be inherited from the model):

```
    "lang": ["en", "dv"]
```

Custom `datalink`

In model-backed mode, CCS auto-generates the data link URL. In noModel mode, you provide it manually:

```
    "table": {
        "noModel": true,
        "datalink": "http://external-api.example.com/api/criminal-
records/6d5c9d34-f80a-4596-ad22-56ca8ffecef1/requests",
        ...
    }
```

This URL is passed through as-is to the frontend.

Custom `crudLink` in Meta

Similarly, the CRUD link must be manually specified:

```
    "meta": {
        "noModel": true,
        "crudLink": "http://external-
api.example.com/api/gapi/CriminalRecordRequest",
        "lang": ["en", "dv"]
    }
```

noModel FilterSection with Inline Schema

When using noModel for the filterSection, you provide `columnsSchema` with the filter columns' definitions directly:

```
    "filterSection": {
        "noModel": true,
        "lang": ["en", "dv"],
        "filters": ["status"],
        "columnsSchema": {
            "status": {
                "key": "status",
                "label": { "dv": "ستۇڊەنٹ", "en": "Status" },
                "lang": ["en", "dv"],
                "inputType": "select",
                "select": {
                    "mode": "self",
                    "items": [
                        { "itemTitleEn": "Draft", "itemTitleDv":
"ڊرافٽ", "itemValue": "draft" },
                        { "itemTitleEn": "Submitted", "itemTitleDv":
"سَبْمِٽڈ", "itemValue": "submitted" }
                    ]
                }
            }
        }
    }
```

noModel Toolbar

Same pattern — provide `columnsSchema` for any filter columns the toolbar needs:

```
    "toolbar": {
        "noModel": true,
        "lang": ["en", "dv"],
```

```
        "title": { "en": "Forms", "dv": "ﷺ" },
        "filters": ["status"],
        "buttons": ["search", "clear"],
        "columnsSchema": {
            "status": {
                "key": "status",
                "label": { "dv": "ﷺ", "en": "Status" },
                "lang": ["en", "dv"],
                "inputType": "select",
                "select": {
                    "mode": "self",
                    "items": [
                        { "itemTitleEn": "Draft", "itemTitleDv":
"ﷺ", "itemValue": "draft" }
                    ]
                }
            }
        }
    }
```

noModel Form

The form component in noModel mode doesn't necessarily need `columnsSchema` because fields are defined explicitly. But you still set `noModel: true` and provide `lang`:

```
    "form": {
        "noModel": true,
        "lang": ["en", "dv"],
        "groups": [ ... ],
        "fields": [ ... ],
        "variables": {}
    }
```

Model-Backed vs noModel Comparison

Aspect	Model-Backed	noModel
Column definitions	From `apiSchema()` on Eloquent model	From inline `columnsSchema`
Data link	Auto-generated `gapi/{Model}?columns=...`	Manual URL string
CRUD link	Auto-generated `gapi/{Model}`	Manual URL string

Aspect	Model-Backed	noModel
`lang`	Can inherit from view	Must be explicit on each component
Relation columns	Resolved via Eloquent relationships	Defined directly in `columnsSchema` with dot-notation keys
Form fields	Auto-generated from `formField: true` columns	Explicitly declared in `fields`
`noModel` flag	Not needed (default `false`)	Required on each component
`columnsSchema`	Not needed	Required on table, toolbar, filterSection

---

15. Validation Rules

The `ViewConfigValidator` validates your view config JSON and reports errors and warnings. Validation runs automatically when `debug_level` is 2+ in `config/uiapi.php`.

Errors (Block the Request)

Rule	What It Checks
`lang` required	Every component must have a `lang` array (either directly or inherited from the view).
`columns` required	Views that reference a `table` component must define `columns`.
`noModel` requires `columnsSchema`	When `noModel: true`, components that need column data (table, toolbar, filterSection) must have `columnsSchema`.
View requires `components`	Every view block must have a `components` object.
Component not found	A component reference in a view must point to an existing root-level key.
`inputType: "select"` requires config	When `inputType` is `"select"`, a `select` or `filterable` config object must exist.
`mode: "url"` requires `url`	Select mode `"url"` requires a valid `url` string.

Rule	What It Checks
Functions require `file` and `function`	Object-style function definitions must have both keys.
Search requires `submitUrl`	Fields with `inputType: "search"` must have a `submitUrl`.

Warnings (Informational)

Rule	What It Checks
`per_page` positive	`per_page` should be a positive integer.
Filter key exists	Filter entries should reference known columns in the schema.
Column exists in schema	`columns` entries should reference valid `apiSchema()` or `columnsSchema` keys.
`displayType` requires config	`displayType: "chip"` or `"select"` should have a matching sub-key or `displayProps`.
`mode: "self"` requires `items`	Self-mode select should have a non-empty `items` array.
`mode: "relation"` requires `relationship`	Relation-mode select should specify the relationship name.
Field group exists	Field `group` values should match one of the `groups[].name` values.
Group name unique	Group names should not be duplicated.
Group title localized	Group titles should be `{en, dv}` objects.
Event handler exists	Field event handler names should exist in the `functions` block.
Column customization key exists	Customized columns should exist in the schema (unless `displayType: "custom"`).
Component config file exists	Referenced components should have matching template files.
Cross-model reference	References to other models' configs generate a "ensure it exists" warning.

---

16. Complete Examples

Model-Backed Example (cform.json)

A full view config for the `CForm` model with table, form, toolbar, filterSection, and meta:

```
    {
        "listView": {
            "components": {
                "table": "CForm/table",
                "form": "CForm/form",
                "toolbar": "CForm/toolbar",
                "filterSection": "CForm/filterSection",
                "meta": "CForm/meta"
            },
            "lang": ["en", "dv"]
        },

        "table": {
            "per_page": 15,
            "actions": {
                "showView": false
            },
            "columns": [
                "id",
                "uuid",
                "ref_num",
                "summary",
                "status",
                "createdby.first_name_div"
            ],
            "columnCustomizations": {
                "New_ref": {
                    "width": "400px",
                    "label": { "en": "New Column", "dv": "ތާވަން ކޮލަމْ" },
                    "columnData": "customeColumnData()",
                    "order": 3,
                    "displayType": "custom"
                },
                "ref_num": {
                    "width": "auto",
                    "align": "center",
                    "sortable": true
                },
                "status": {
                    "displayType": "chip",
                    "chip": {
                        "draft": {
                            "size": "sm",
                            "label": { "en": "Draft", "dv": "ڊﷺ" },
                            "color": "secondary",
                            "prependIcon": "document-text"
                        },
                        "submitted": {
                            "size": "sm",
                            "label": { "en": "Submitted", "dv": "ﷺ"
},
                            "color": "primary",
                            "prependIcon": "paper-airplane"
                        },
```

```
                        "rejected": {
                            "size": "sm",
                            "label": { "en": "Rejected", "dv":
"ބާބަލާނެ" },
                            "color": "error",
                            "prependIcon": "x-circle"
                        },
                        "accepted": {
                            "size": "sm",
                            "label": { "en": "Accepted", "dv":
"ބަދަލުކުރެވުނު" },
                            "color": "success",
                            "prependIcon": "check-circle"
                        }
                    },
                    "sortable": true,
                    "inlineEditable": true
                }
            },
            "functions": {
                "customeColumnData": {
                    "file": "misc.js",
                    "function": "customeColumnData"
                }
            },
            "delete": {
                "title": { "en": "Confirm Deletion", "dv": "ހުވަދުވުން ބަހައްޓ�"
},
                "subtitle": {
                    "en": "Are you sure you want to delete this item?",
                    "dv": "..."
                },
                "message": {
                    "ref_num": { "en": "ref", "dv": "ތަން" },
                    "uuid": { "en": "UUID", "dv": "ހުވަން" }
                },
                "crudLink": "gapi/CForm"
            }
        },

        "form": {
            "createTitle": { "en": "Create Form", "dv": "ތަން" },
            "editTitle": { "en": "Edit Form", "dv": "ބदަލުކުރ" },
            "banner": {
                "title": { "en": "Form Details", "dv": "ފورމުގެ ތަފْصِيلْ"
},
                "message": {
                    "en": "Please fill out the form below. Fields
marked with * are required.",
                    "dv": "..."
                },
                "color": "info",
                "icon": "information-circle",
                "variant": "tonal"
            },
            "groups": [
                {
                    "name": "Search",
```

```
                    "title": { "en": "Search", "dv": "ڑُعَرَ" },
                    "numberOfColumns": 2
                },
                {
                    "name": "BasicInfo",
                    "condition": "SearchResults || formType ===
'edit'",
                    "title": { "en": "Basic Information", "dv": "ضَدْوُوَدَةُ"
},
                    "numberOfColumns": 3
                },
                {
                    "name": "Submission",
                    "condition": "SearchResults || formType ===
'edit'",
                    "title": { "en": "Submission", "dv": "ءِعْبَرْ هَوْرْسِدةْ" },
                    "numberOfColumns": 2
                }
            ],
            "fields": [
                {
                    "key": "nid_pp",
                    "group": "Search",
                    "disabled": "mode == 'edit' ? true : false",
                    "columnSpan": 2,
                    "inputType": "search",
                    "submitUrl": "gapi/person/",
                    "submitMethod": "get",
                    "NotDbField": true,
                    "lang": "en",
                    "submitParam": "lang=${language}&filter",
                    "events": { "results": "handleSearchResults" }
                },
                {
                    "key": "ref_num",
                    "disabled": "mode == 'edit' ? true : false",
                    "group": "BasicInfo",
                    "rules": "[rules.required]"
                },
                { "key": "client_ref_number", "group": "BasicInfo" },
                { "key": "qazziyya_number", "group": "BasicInfo" },
                {
                    "key": "summary",
                    "inputType": "textarea",
                    "group": "BasicInfo",
                    "columnSpan": 3,
                    "hidden": false
                },
                { "key": "sender_type", "group": "Submission" },
                {
                    "key": "entry_type",
                    "group": "Submission",
                    "inputType": "select",
                    "mode": "url",
                    "url": "http://v2.govapi.pgo.mv/api/entry-types",
                    "itemTitle": "itemTitleDv",
                    "itemValue": "itemValue"
                },
                { "key": "total_pages", "group": "Submission" },
                { "key": "status", "group": "Submission" },
```

```
                {
                    "key": "submitted_at",
                    "group": "Submission",
                    "format": "dd/MM/yyyy"
                },
                {
                    "key": "remarks",
                    "group": "Submission",
                    "placeholder": "asdf"
                },
                {
                    "key": "file_upload",
                    "label": { "en": "Upload File", "dv": "ف،ل،ف ،ت،ث،ت،ي،"
},
                    "inputType": "file",
                    "columnSpan": 3
                }
            ],
            "variables": { "draftingEnabled": true, "SearchResults":
true },
            "functions": {
                "handleSearchResults": {
                    "file": "misc.js",
                    "function": "handleSearchResults"
                }
            }
        },

        "toolbar": {
            "title": { "en": "Forms", "dv": "ت،ف،ر،ل،ف،" },
            "filters": ["status", "search"],
            "buttons": ["search", "clear"],
            "columnCustomizations": {
                "search": {
                    "key": "ref_num",
                    "label": { "en": "Search by ID", "dv": "،ت،ي، ،ي،ث، ،ث،ر،"
},
                    "color": "primary",
                    "prependIcon": "search"
                }
            }
        },

        "filterSection": {
            "filters": ["status"],
            "columnCustomizations": {
                "status": {
                    "width": "180px",
                    "displayType": "chip",
                    "sortable": true,
                    "inlineEditable": true,
                    "inputType": "select",
                    "select": {
                        "type": "select",
                        "mode": "self",
                        "items": [
                            { "itemTitleEn": "Draft", "itemTitleDv":
"،ث،ر،ف،ق،", "itemValue": "draft" },
                            { "itemTitleEn": "Submitted",
```

```
"itemTitleDv": "ﷺ", "itemValue": "submitted" },
                            { "itemTitleEn": "Rejected", "itemTitleDv":
"ﷺ", "itemValue": "rejected" },
                            { "itemTitleEn": "Accepted", "itemTitleDv":
"ﷺ", "itemValue": "accepted" }
                        ]
                    }
                }
            }
        },

        "meta": {}
    }
```

noModel Example (criminalrecord.json)

A full view config for external API data with no Eloquent model:

```
    {
        "listView": {
            "noModel": true,
            "components": {
                "table": "criminalrecord/table",
                "form": "criminalrecord/form",
                "filterSection": "criminalrecord/filterSection",
                "toolbar": "criminalrecord/toolbar",
                "meta": "criminalrecord/meta"
            }
        },

        "table": {
            "noModel": true,
            "lang": ["en", "dv"],
            "per_page": 7,
            "datalink": "http://external-api.example.com/api/criminal-
records/.../requests",
            "delete": {
                "title": { "en": "Confirm Deletion", "dv": "..." },
                "subtitle": { "en": "Are you sure?", "dv": "..." },
                "message": {
                    "ref_num": { "en": "ref", "dv": "ﷺ" }
                }
            },
            "columnsSchema": {
                "id": {
                    "hidden": false,
                    "key": "id",
                    "label": { "dv": "ﷺ", "en": "Id" },
                    "lang": ["en", "dv"],
                    "type": "number",
                    "displayType": "text",
                    "formField": false,
```

```
                    "sortable": true
                },
                "ref_num": {
                    "hidden": false,
                    "key": "ref_num",
                    "label": { "dv": "ﷺ", "en": "Reference
Number" },
                    "lang": ["en", "dv"],
                    "type": "text",
                    "displayType": "text",
                    "formField": false,
                    "sortable": true
                },
                "person_name": {
                    "hidden": false,
                    "key": "person_name",
                    "label": { "dv": "ﷺ", "en": "Person Name" },
                    "lang": ["en", "dv"],
                    "type": "text",
                    "displayType": "text",
                    "formField": false,
                    "sortable": true
                },
                "status": {
                    "hidden": false,
                    "key": "status",
                    "label": { "dv": "ﷺ", "en": "Status" },
                    "lang": ["en", "dv"],
                    "type": "text",
                    "displayType": "text",
                    "formField": false,
                    "sortable": true
                }
            },
            "columns": ["id", "ref_num", "person_name", "status"],
            "columnCustomizations": {
                "ref_num": {
                    "width": "200px",
                    "align": "center",
                    "sortable": true
                },
                "status": {
                    "width": "180px",
                    "displayType": "chip",
                    "chip": {
                        "draft": {
                            "size": "sm",
                            "label": { "en": "Draft", "dv": "ﷺ" },
                            "color": "secondary",
                            "prependIcon": "file"
                        },
                        "completed": {
                            "size": "sm",
                            "label": { "en": "Completed", "dv": "ﷺ"
},
                            "color": "success",
                            "prependIcon": "check-circle"
                        }
                    },
```

```
                    "sortable": true,
                    "inlineEditable": true
                }
            }
        },

        "form": {
            "noModel": true,
            "lang": ["en", "dv"],
            "groups": [
                {
                    "name": "PersonalDetailsEng",
                    "title": { "en": "Check if the person is
Maldivian", "dv": "..." },
                    "numberOfColumns": 3
                },
                {
                    "name": "PersonalDetails",
                    "title": { "en": "Person Details", "dv": "ھۮ ۋ ۾ ۽ ۾ ۋ" },
                    "numberOfColumns": 3
                }
            ],
            "fields": [
                {
                    "key": "ref_number_check",
                    "inputType": "checkbox",
                    "columnSpan": 3,
                    "group": "PersonalDetailsEng",
                    "label": { "en": "Check if the person has a
reference number", "dv": "..." },
                    "color": "primary"
                },
                {
                    "key": "passport_id",
                    "columnSpan": 3,
                    "condition": "!ref_number_check",
                    "group": "PersonalDetails",
                    "label": { "en": "Passport Number", "dv": "ۋ ۾ ۾ ۾ ۾ ۾ ۾
ۋ ۾ ۾ ۾ ۾ ۾ ۾ ۾" }
                },
                {
                    "key": "person__id",
                    "columnSpan": 3,
                    "condition": "ref_number_check",
                    "group": "PersonalDetails",
                    "label": { "en": "National ID", "dv": "ۋ ۾ ۾ ۾ ۾ ۾ ۾ ۾ ۾" }
                },
                {
                    "key": "person_name",
                    "columnSpan": 3,
                    "group": "PersonalDetails",
                    "label": { "en": "Person Name", "dv": "ۋ ۾ ۾ ۾ ۾ ۾ ۾ ۾" }
                },
                {
                    "key": "details",
                    "columnSpan": 3,
                    "group": "PersonalDetails",
                    "label": { "en": "Reason for checking", "dv":
```

```
"ކުރިން ބެލެހެއްޓެން ހުރިކަން" }
                }
            ],
            "variables": {}
        },

        "meta": {
            "noModel": true,
            "crudLink": "http://external-
api.example.com/api/gapi/CriminalRecordRequest",
            "lang": ["en", "dv"]
        },

        "filterSection": {
            "noModel": true,
            "lang": ["en", "dv"],
            "filters": ["status"],
            "buttons": ["submit", "clear"],
            "columnsSchema": {
                "status": {
                    "key": "status",
                    "label": { "dv": "ސ્ટેટસ્", "en": "Status" },
                    "lang": ["en", "dv"],
                    "inputType": "select",
                    "select": {
                        "mode": "self",
                        "items": [
                            { "itemTitleEn": "Draft", "itemTitleDv":
"ڊ۾ڙڻ", "itemValue": "draft" },
                            { "itemTitleEn": "Submitted",
"itemTitleDv": "ހުށަހެޅި", "itemValue": "submitted" },
                            { "itemTitleEn": "Completed",
"itemTitleDv": "ހިންދެވިއްޖެ", "itemValue": "completed" }
                        ]
                    }
                }
            }
        },

        "toolbar": {
            "noModel": true,
            "lang": ["en", "dv"],
            "title": { "en": "Criminal Records", "dv": "ތާރީ ފެންވަރ" },
            "filters": ["status"],
            "buttons": ["search", "clear"],
            "columnsSchema": {
                "status": {
                    "key": "status",
                    "label": { "dv": "ستەٹەس", "en": "Status" },
                    "lang": ["en", "dv"],
                    "inputType": "select",
                    "select": {
                        "mode": "self",
                        "items": [
                            { "itemTitleEn": "Draft", "itemTitleDv":
"ڊ۾ڙڻ", "itemValue": "draft" },
                            { "itemTitleEn": "Completed",
```

```
"itemTitleDv": "سرجوڤۆر", "itemValue": "completed" }
                        ]
                    }
                }
            }
        }
    }
```

---

17. Quick Checklist

Model-Backed Config

- File named correctly (lowercase, no separators, `.json`)
- At least one view block with `"components"` map
- Component references point to existing root-level keys
- `lang` defined (on view or on each component)
- `columns` array on the table component with fields to display
- `columnCustomizations` for display overrides (chips, dates, widths)
- `filters` array on toolbar/filterSection if you want filter UI
- `groups` and `fields` defined for the form
- `functions` block if using external JS or event handlers
- All `{dv}` labels filled in (not left as "TODO")
- `delete` config on table if delete action is enabled
- `meta` component included (for CRUD link)

noModel Config

All the above, plus:

- `"noModel": true` on every component definition
- `columnsSchema` on table, toolbar, and filterSection
- `lang` explicitly on every component (no model to inherit from)
- `datalink` manually set on table (custom URL)
- `crudLink` manually set on meta (custom URL)
- Form fields fully declared (no auto-generation from schema)

Internal Keys (Stripped from Payload)

These keys are consumed by CCS during processing and never appear in the API response:

Key	Purpose
`columnCustomizations`	Column display overrides
`columnsSchema`	noModel column definitions
`noModel`	noModel flag
`columns`	Column ordering
`per_page`	Pagination size
`filters`	Allowed filter columns
`lang`	Language support